University of Liverpool
Department of Computer Science
COMP702 MSc Computer Science Project

Kodro

An Offline, Self-Refining Platform for Designing Robots and Validating Their Behaviour in Realistic Simulation

---

Vaibhav Lalwani
Student ID: 201979723

Supervisor: Keith Dures

Submitted in partial fulfilment of the requirements for the degree of
Master of Science in Computer Science

11 September 2026

Word count excludes the title page, contents, references and appendices.

Declaration

I confirm that this dissertation is my own work and that it has not been submitted, in whole or in part, for any other degree or qualification. Where I have drawn on the work or ideas of others, the source is acknowledged in the text and listed in the references. The software described here was built by me over the project period and is held in a version controlled repository, so every technical claim in this document can be checked against the code at the tag it describes.

Use of generative artificial intelligence. Generative artificial intelligence assisted the development of the software and the drafting of this document, used in line with University guidance on acknowledging such assistance. I directed the work, made every design decision, and verified every result reported here. All technical claims describe the working system as built and tested. Every figure that reports a measurement is reported as measured, with the method and its limits stated plainly. No result from a human study is reported, because that study has not yet been run, and the text says so wherever it is relevant rather than implying evidence that does not exist. The references were checked at the time of writing.

Ethics. The work to date uses simulated personas only, with no human participants and no personal data (ethics data category A0, participant category 2). Any later study with real users will be agreed with my supervisor and run under appropriate informed consent before it begins.

Abstract

Many people have an idea for a robot and no safe, cheap way to try it. Buying parts, wiring a board and writing control code is slow, and the first time a real robot meets the real world it tends to drive into something. Kodro is an offline desktop platform that closes that gap. A non-expert designs a custom robot from real parts, programs it in Python, in blocks, or by voice with a grounded local assistant and an automatic code reviewer, and validates its behaviour in a realistic simulated world before building any hardware. The robot the user assembles drives the simulation, so changing the build changes the behaviour: a heavier robot accelerates more slowly and drains its battery faster, a stronger motor lifts the top speed, and a sensor that is not fitted withholds its command from every way of programming the robot.

A small language model, run only on the local machine through Ollama, grounds its suggestions in that build and refines its advice from accumulated use through a reflection loop and a reusable skill library, without retraining the model and without reaching the internet. When no model is present the platform falls back within a fixed time budget to a deterministic rule engine, so it stays fully usable with the network off. A deterministic validator, not the model, always has the final word on whether a program is safe to run.

This dissertation states the problem and the aims, reviews the literature the design follows from, sets out the requirements and the layered architecture, describes the implementation against the real codebase, and reports an honest evaluation. The interpreter and its kinematics pass a functional quality harness at twenty one of twenty one, the Python engine carries a suite of more than eight hundred tests at around eighty six percent coverage, and a formative review across simulated personas traces the build from a mean of 5.86 out of ten to 7.36 as defects were fixed. The contribution is not a new algorithm. It is a working demonstration that retrieval grounding, a sandboxed simulation gate, an experience memory, a skill library and domain randomisation combine into an honest design and validation platform that runs entirely on one machine, together with a measured account of how far offline self-refinement can go when changing the model weights is off the table.

Acknowledgements

I thank my supervisor, Keith Dures, for steady guidance and for keeping the scope honest. I thank the maintainers of the open-source libraries this work stands on, named in the implementation chapter. The framing of the project owes a debt to a long line of work on learning by building, on grounding language models in what a system can actually do, and on closing the gap between simulation and reality, all of which is acknowledged in the references.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context and motivation

A robot is one of the few pieces of software that can hurt someone if it is wrong. A spreadsheet that miscalculates wastes time. A control program that misjudges a distance drives a machine into a wall, or into a person. This is the reason the gap between an idea for a robot and a working robot is wide, and it is why the gap is expensive to cross. Parts cost money, wiring takes time and care, and the first contact between a freshly written control program and the physical world is usually a small disaster. People who are not professional roboticists, the makers, the hobbyists, the students, the curious, feel this barrier most sharply. They have the idea and the willingness, and they lack a safe and cheap place to try the idea before they commit money and hardware to it.

The obvious answer is simulation. Try the robot in software first, watch it behave, fix what is wrong, and only then build it. This is sound, and it is exactly what professional roboticists do with heavyweight simulators. The difficulty is that those simulators are heavyweight in every sense. They assume a workstation, a graphics card, an internet connection for assets and accounts, and an operator who already knows robotics. They are built for the expert who is refining a known design, not for the non-expert who is exploring whether an idea is worth pursuing at all. A person who has never wired a board cannot sit down in front of an industrial simulator and get useful work done in an afternoon.

There is a second answer that has become tempting in the last few years, which is to ask a large language model to write the control code. This works until it does not, and the way it fails is dangerous in this setting. A language model produces plausible text, and plausible control code that calls a sensor the robot does not have, or that assumes a capability the hardware lacks, reads exactly like correct control code to a beginner. The output looks authoritative and is sometimes wrong, and the person least able to tell the difference is the very person the tool is meant to help.

Kodro is built where these three observations meet. It gives a non-expert a place to design a robot and validate its behaviour in software before spending anything on hardware. It does this on one ordinary laptop, offline, with no account and no cloud subscription, so cost and privacy are not barriers and a locked-down or disconnected machine is not an obstacle. And it treats the language model as a useful but untrusted assistant rather than an oracle, grounding every suggestion in the robot the user actually built and placing a deterministic simulation gate between any generated program and the world.

1.2 Problem statement

The problem this project addresses can be stated in one sentence. How can a single, offline, free desktop application let a capable non-expert design a custom robot, program its behaviour, and validate that behaviour in a realistic, agent-populated simulation before building any hardware, while using a local language model for help in a way that is genuinely useful and genuinely safe?

Three words in that sentence carry most of the weight. *Realistic* means the simulation has to be worth trusting: the robot must behave in a way that reflects its build, the world must contain

things that move and respond, and a run must report what happened across varied conditions rather than one tidy outcome. *Offline* means a hard constraint, not a preference: no cloud service, no account, no paid interface, no data leaving the machine, which rules out the easy path of calling a large hosted model and forces every capability onto local resources. *Safe* means that the help the tool gives cannot quietly produce a program that would drive a real machine into a person, which makes the relationship between the model and the validator the central design question rather than an afterthought.

1.3 Aims and objectives

The aim of the project is a working offline platform on which a non-expert designs a robot, programs it, and validates that behaviour in a realistic, agent-populated simulation before building it. Each objective below carries a plain test of done, so progress is judged rather than asserted.

1.3.1 Core objectives

These define the artefact. The platform must achieve all of them to count as a working system.

O1. **Specification-driven design.** Let the user design a robot from a board, sensors and actuators, where the specification drives the simulation. *Done when* changing the build changes behaviour: a heavier robot accelerates more slowly and drains its battery faster, a stronger motor raises the top speed, and a fitted sensor unlocks its command while an absent one withholds it from every way of programming the robot.

O2. **Safe execution and scoring.** Execute the robot's real program in a sandbox that blocks file, network and process access, and score every run against the scenario's success criteria. *Done when* a hostile program cannot reach the disk and a correct one is scored correctly.

O3. **Realistic, randomised validation.** Run the robot in a world with terrain and moving agents, across repeated randomised runs. *Done when* one program runs over several seeds and returns a report of successes, collisions, time to goal and battery used.

O4. **Grounded local assistance with a guaranteed fallback.** Run the assistant on a local model only, grounded in the robot's specification, and fall back to deterministic rule-based behaviour within a fixed time budget when no model is present. *Done when* the platform is fully usable with the network off and no model installed, and when the model cannot call a part the build lacks.

O5. **Refinement from use without retraining.** Refine suggestions from accumulated use through reflection retrieval and a reusable skill library. *Done when*, across a session, the system reuses a stored reflection or skill and the median iterations to a working design fall.

1.3.2 Secondary objectives

These raise the quality of the artefact once the core holds.

O6. **Three ways to program, one meaning.** Provide a text editor, a blocks palette that compiles to the same program, and a voice route, so that the way the user expresses a behaviour does not change what the behaviour means.

O7. **Legible realism on a normal laptop.** Render the worlds and the robot with enough fidelity to read as believable in a screenshot, while staying smooth on a machine with no discrete graphics card.

O8. **An honest record.** Build the system as small, independently tested changes behind a continuous integration gate, and keep a decision log and a test-evidence log, so the work is reproducible and the dissertation can be checked against it.

1.3.3 Out of scope

Two things are deliberately not attempted, and saying so keeps the claims honest. Kodro does not aim at production-grade physical accuracy; its motion is a believable kinematic model, not a rigid-body solver, and the discussion chapter is explicit about what that does and does not buy. Kodro also does not aim to replace the established heavyweight simulators such as Isaac Sim, Gazebo, Webots or MuJoCo; the useful relationship with those tools is a research bridge on the roadmap, not a claim of equivalence.

1.4 Contributions

This project contributes a complete, working artefact and the engineering record behind it. Concretely it delivers:

- a desktop application, fully offline, on which a non-expert assembles a robot from real parts and the build derives the robot's mass, top speed, battery life and the exact set of commands it can run;
- three ways to program one robot, a Python subset interpreter, a blocks palette that compiles to the same Python, and a voice route, all driving a single command surface so the meaning of a behaviour does not depend on how it was written;
- a grounded local assistant and an automatic code reviewer that suggest and check code only against the parts the robot carries, backed by a deterministic rule engine that takes over within a fixed time budget when no model is installed, so the model can help but never has the last word on safety;
- a validation layer that runs a program in a sandboxed world among moving agents and reports what happened, with a self-refinement memory of reflections and reusable skills that informs later sessions without retraining any model; and
- a disciplined development process, recorded in a decision log and a running test-evidence log and gated by continuous integration, that makes the work reproducible and gives the critical reflection in this dissertation a factual basis.

The contribution is not a new algorithm. Retrieval grounding, a sandboxed gate, an experience memory, a skill library and domain randomisation are each known. The contribution is the demonstration that these parts combine into an honest design and validation platform that runs entirely on one machine, and a measured account of how far offline self-refinement can go when changing the model weights is off the table.

1.5 Report structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature the design follows from, across learning by building, the gap between simulation and reality, and the grounding and honest framing of language models. Chapter 3 sets out the requirements, the intended users including who the tool is not for, and the analysis that shaped the design. Chapter 4 describes the layered architecture and the key design decisions, chief among them the robot specification as the single source of truth and the deterministic gate that stands between a generated program and the world. Chapter 5 describes the implementation against the real code, the techniques that recur, and the engineering challenges that were met and fixed. Chapter 6 reports the evaluation, separating what was measured from what is planned, and stating the threats to validity plainly. Chapter 7 discusses the honest limitations of the system. Chapter 8 concludes and sets out the future work. The references and appendices follow.

Chapter 2

Background and Related Work

This chapter situates Kodro against three lines of work rather than placing it beside them. The design is a consequence of these lines, so the chapter reads each one for what it implies for an offline design and validation platform, and ends by naming the niche the surveyed work leaves open.

2.1 Learning by building

The oldest of the three lines is constructionism, the durable finding that people learn most deeply when they build something external that they can watch behave and then debug (Papert, 1980). The important word is *watch*. A learner who can see the consequence of a change, and can change it again, is in a tight feedback loop that abstract instruction cannot match. Kodro is built around exactly this loop. The user assembles a robot, writes a behaviour, runs it, sees the robot behave in a world, and adjusts. The blocks palette compiles to the same Python the text editor runs, so the move from blocks to text is a change of surface and not a change of tool, and the learner is never asked to throw away what they understood in order to progress. This is a deliberate echo of the constructionist position that the medium should not get in the way of the idea.

2.2 Trusting a simulator: the gap between simulation and reality

The second line is the question of why a simulator is worth trusting at all. The central and well-documented problem in robotics is the gap between simulation and reality, the drop in performance when a behaviour tuned in simulation meets the physical world. The accepted response is not the pursuit of a perfect simulator, which is unattainable, but domain randomisation: vary the friction, the mass, the sensor noise and the scene across many runs, so that a behaviour which survives the spread is likely to survive reality too (Tobin et al., 2017). The insight is that variability is the asset, not the enemy. A program that only works on one tidy layout has learned the layout; a program that works across a randomised spread has learned something more general.

Kodro adopts this directly. A design is validated across randomised seeds with varied friction, mass and sensor noise, and a run reports the spread and not only the average. The worlds are populated with agents that move and respond, traffic that keeps to one-way lanes and brakes for the robot, pedestrians that use the crossing, so that a self-driving car or a home robot is tested among things that move rather than on an empty plane. The design follows the literature here because the literature is right: the value of a simulator is not its prettiness but the honesty of the spread it reports.

2.3 The model: hallucination, grounding, and honest refinement

The third line concerns the language model itself, and it justifies both the offline choice and the careful framing of refinement. Two findings shape the design. The first is that hallucination is intrinsic to these models rather than a defect that a future version will simply remove (Huang et al., 2023). In generated code this surfaces as plausible but wrong calls (Lee et al., 2025),

which is precisely the dangerous failure when the output is control code that a beginner cannot evaluate. The second is the standard mitigation: ground generation in retrieved, authoritative material rather than in the model's memory (Lewis et al., 2020). For robots, the strongest form of grounding is to constrain the model to compose a known, safe set of capabilities rather than to invent them, in the spirit of programs written against a fixed robot interface (Liang et al., 2023; dos Santos et al., 2026) and of grounding instructions in what the robot can actually do (Ahn et al., 2022).

Kodro grounds every suggestion in the robot the user assembled, so the model cannot call a sensor that is not fitted. This grounding is also why no cloud is needed: the material the model is grounded in, the build, lives on the machine, so the help can be local. On refinement over time the design is deliberately careful, because the honest state of the art is that genuine weight-level self-evolution is unsolved (Tao et al., 2024). Kodro therefore does not claim it. Instead it refines at the level of the system, through a verbal reflection loop (Shinn et al., 2023) and a growing library of behaviours that worked (Wang et al., 2023), composed for new designs without retraining, mirroring recent experience-driven self-evolving agents that improve from their own history without changing weights (Wu et al., 2025). Where several agents cooperate, naive arrangements fail in characteristic ways (Cemri et al., 2025), which is why in Kodro a deterministic check, not another model, always has the final word, with an automatic reviewer flagging suspect code before that, following recent automated code-review practice (Sun et al., 2025), and deterministic static analysis can catch hallucinated code outright (Khati et al., 2026).

2.4 Small local models

A fourth, narrower body of recent work makes the offline choice practical rather than merely principled. Small open-weight models, in the one to three billion parameter range, paired with retrieval, now give reliable and low-hallucination help offline (Reza et al., 2025), and onboard small models are being shown to drive robot task planning and code generation on constrained hardware (Wang et al., 2026). This matters because the size is the binding constraint for the target machine. A model in this range is the largest class that still answers at an interactive pace on a mainstream laptop with no discrete graphics card, where a seven billion model would demand a card a school or home laptop rarely has. The honest limit, discussed again in the evaluation and the conclusion, is that a small local model buys domain grounding and a reliable output format, not frontier capability, so the design leans on grounding and the simulation gate and never on the model being clever.

2.5 The niche

The surveyed work leaves a specific gap open. There are excellent heavyweight simulators for experts, excellent block-based environments for children, and a growing literature on grounding and on honest self-evolution, but there is no single offline desktop tool that lets a capable non-expert design a robot, program it three ways against one meaning, and validate its behaviour among moving agents across a randomised spread, with a local model that is grounded hard enough to be safe and a deterministic gate that has the final word. Kodro is built to fill exactly that gap, and the rest of this dissertation describes how, and reports how far it gets.

Chapter 3

Requirements and Analysis

This chapter turns the aims of Chapter 1 into requirements that the design can be measured against. It begins with the intended user, because the user decides almost everything that follows, and it is deliberate about who the tool is not for. It then states the functional and non-functional requirements, and analyses the central tension that shaped the whole system, which is the relationship between a helpful model and a trustworthy result.

3.1 The intended user, and who it is not for

Kodro is built for a capable non-expert adult. The picture to hold in mind is a person who can think clearly, can follow a structured tool, and has an idea for a robot, but who does not have a lab, a kit, a budget for parts, or training in robotics. This is the maker who wants to know whether a companion robot idea is worth pursuing, the student who wants to test a self-driving behaviour without a car, the hobbyist who wants to see a rover handle rough ground before buying the rover. The tool earns its place by letting that person get from an idea to a validated behaviour in an afternoon, on the laptop they already own, without spending anything and without an account.

It is equally important to say who the tool is not for, because a tool that claims everyone as its user serves no one. Kodro is not for a professional roboticist refining a known design to deployment tolerances; that person needs rigid-body physics and the established heavyweight simulators, and Kodro does not pretend to compete with them. It is not a children's coding toy; the framing, the language and the expectations assume an adult who wants a serious result, not a gamified first lesson. It is not for a team that needs to collaborate over a network on a shared design; the offline, single-machine constraint that makes Kodro safe and private also makes it deliberately solitary. And it is not for anyone who needs the model to be brilliant, because the model is small, local and grounded on purpose, and the design never asks it to be clever. Naming these non-users is not modesty. It is the discipline that keeps the rest of the requirements honest.

3.2 Functional requirements

The functional requirements fall into five groups, matching the core objectives. They are stated as testable capabilities.

3.3 Non-functional requirements

The non-functional requirements are where the project's character lives, because the offline constraint touches every one of them.

- **Offline by construction.** No cloud service, no account, no paid interface, and no data leaving the machine on any path. The only networked call permitted is to a local model server on the loopback address, and the platform must be fully functional with that absent.
- **Runs on a mainstream laptop.** The worlds and the robot must render smoothly on a machine with no discrete graphics card, which bounds the visual approach to procedural geometry and a measured lighting model rather than heavyweight assets.

ID	Requirement	Test of satisfaction
F1	Assemble a robot from parts	The user picks a chassis, a board, sensors and actuators, and the build derives mass, top speed, battery capacity and the set of runnable commands.
F2	Specification drives behaviour	Changing the build changes the simulation: a heavier robot accelerates more slowly and drains battery faster; a stronger motor lifts top speed.
F3	Command gating	A command appears only if the part it needs is fitted, and disappears from the editor, the blocks palette, the voice route and the assistant when the part is removed.
F4	Three ways to program	A text editor, a blocks palette that compiles to the same program, and a voice route, all driving one command surface.
F5	Safe execution	A program runs in a sandbox that blocks file, network and process access, and a hostile program cannot reach the disk.
F6	Scenario scoring	A run is scored against the scenario success criteria, returning a pass or fail, a score, and a per-criterion reason.
F7	Randomised validation	One program runs across several seeds with varied friction, mass and sensor noise, returning a report of successes, collisions, time and battery.
F8	Grounded assistant	The local model suggests code only against fitted parts, and refuses and points back to the build when asked for a part that is absent.
F9	Deterministic fallback	With no model installed, a rule engine returns code and checks within a fixed time budget, and the platform stays fully usable.
F10	Self-refinement memory	After a run the system records a reflection and may save a skill, and retrieves similar past material to inform later suggestions.

Table 3.1. Functional requirements and their tests of satisfaction.

- **Deterministic where it matters.** The simulation, the scoring and the safety checks must be reproducible, so they can be tested without a display and so a result can be trusted to mean the same thing twice.
- **Safe by default.** The system must fail toward safety. An absent model, a missing audio device, a headless environment, or a hostile program must each produce a clear, contained outcome rather than a crash or an unguarded action.
- **Legible.** A non-expert must be able to read the program surface and understand it, which is why the command API is a small set of plainly named functions rather than an object hierarchy.

3.4 The central analysis: a helpful model against a trustworthy result

The hardest requirement to satisfy is not any single item in the tables. It is the tension between two of them. Requirement F8 wants the model to be helpful, and helpfulness pushes toward letting the model write whatever code seems to fit. Requirement F5 and the safety non-functional want the result to be trustworthy, and trust pushes toward letting nothing run that has not been

checked. A naive design resolves this tension in the model's favour and ships plausible but wrong control code to a beginner. A timid design resolves it in the validator's favour and produces a tool too rigid to help.

The analysis that shaped Kodro refuses to resolve the tension in either direction and instead separates the two concerns by authority. The model is allowed to be helpful, to draft, to suggest, to explain, but it is never allowed to be the thing that decides a program is safe to run. That decision belongs to deterministic machinery: the grounding that will not let a suggestion name a missing part, the sandbox that blocks unsafe operations before execution, the automatic reviewer that flags suspect code, and the simulation gate that runs the program in a world and scores it before any hardware is involved. The model proposes; the deterministic layer disposes. This single principle, that the model may generate but the validators have final authority, is the spine of the architecture in the next chapter, and the evaluation returns to it to ask whether it actually holds in practice.

Chapter 4

Design and Architecture

This chapter sets out the design that the requirements imply. Every claim here corresponds to code in the repository, and the rationale for the non-obvious choices is recorded contemporaneously in the project's decision log. The chapter moves from the overall shape down to the parts that carry the most weight: the robot specification as the single source of truth, the trace-driven core that makes everything testable, the safety machinery, and the grounded assistant with its deterministic fallback.

4.1 Design goals

Four goals shaped the architecture, in priority order. First, **legibility**: a non-expert must be able to read the program surface and understand it. Second, **determinism and testability**: the simulation, scoring and safety checks must be reproducible and testable without a display. Third, **strict offline operation**: no network dependency anywhere on the critical path. Fourth, **separation of the user's program from the engine internals**, so the user works against a tiny, safe surface and the engine can change underneath without breaking their mental model.

4.2 Technology choices and their rationale

The platform is written in Python 3.12 behind a desktop web view, and each choice follows from the offline, low-resource constraint rather than from fashion. The interface is authored in React but precompiled to a single bundle, so the panelled layout and the modern look are kept while the cost of a runtime compiler is paid once at build time and never on the user's machine. The three-dimensional world uses a copy of the Three.js library held inside the project, core only, so the graphics work with the network off and with no asset pipeline. The user's designs and the system's accumulated record are stored in SQLite, a single standard-library file with no server, so a backup is a file copy and nothing leaves the machine. The assistant calls a local model served by Ollama, which installs as one small application and keeps a model warm with a single command a non-technical user can follow. The desktop shell is pywebview, which renders the vendored interface in a native window using the platform web view, with a Tkinter interface retained as a guaranteed-offline fallback for machines without a modern web view runtime.

4.3 Layered architecture

The system is organised in layers with a one-directional dependency flow, so that each layer depends only on the ones below it. Figure 4.1 shows the design surface where this begins, the Robot Lab, and the layering is as follows.

4.4 The robot specification as the single source of truth

The most important design decision in Kodro is that the robot is not cosmetic. Its specification is a single object that the rest of the system reads, and it is the reason changing the build changes the behaviour. When the user assembles a robot, a derive step computes from the chosen parts a structured specification: the chassis type, the board, the fitted sensors and actuators, and from

Layer	What it is	Where it lives
Program surface	The Python subset, the blocks, the voice route	`interpreter.js`, blocks palette, voice parser
Command registry	The single set of commands the build allows	derived from the robot specification
Robot specification	The single source of truth for the build	`RobotLab.jsx`, the derive step
Worlds and motion	City, room and terrains, the natural-motion tick	`Viewport3D.jsx`, `terrains.jsx`, `agents.jsx`
Validation and memory	Scoring, randomised runs, reflections, skills	`memory.jsx`, the Python grader
Python engine	The metres world, physics wrapper, public API	`engine/`, `rover_api.py`

Table 4.1. The layered architecture and where each layer lives in the codebase.

those a mass in kilograms, a top speed, an acceleration, a turn rate, a battery capacity and a drain rate, a friction profile, and crucially a list of supported commands.

That last field is what makes the design honest. The command registry is generated from the specification, not hand-written per panel. A build without an ultrasonic sensor has no distance-reading command in the registry, so the text editor will not accept it, the blocks palette does not offer it, the voice route will not parse to it, and the grounded assistant cannot suggest it. There is no second place where commands are defined and could drift out of step, because there is only one registry and every surface reads it. This is the structural guarantee behind requirement F3, and it is the mechanism by which the model is grounded: the model is told the registry, and the registry is exactly what the build allows.

The physical fields drive the motion. A heavier robot has a lower acceleration and a faster battery drain, a stronger motor raises the speed cap, and the friction profile changes how the robot holds the ground. The point is that these are not separate sliders the user tunes; they fall out of the parts the user chose, so the design loop is to change the robot, not to change a number, and to watch the behaviour change as a consequence.

4.5 The trace-driven core

A single abstraction underpins scoring, hinting and replay: the trace. Every call the running program makes to the command surface appends an event that records the call, its arguments, its result, a snapshot of the robot's state, and any collision. From this one trace three features are derived without duplicating the representation of what the robot did. Scoring compares aggregates of the trace, the samples collected, the collisions, the battery used, the distance, the final position, against the scenario's declared success criteria, and an analysis of the source against the allowed constructs, to yield a pass or fail, a score, and a per-criterion reason. Hinting runs a set of rule predicates over the same events plus the score and surfaces the first that matches. Replay reconstructs the run from the serialised trace and scrubs through it.

The value of this choice is that the decision logic is pure. The scorer is a function from a scenario, a trace and the source to a result, with no input or output and no hidden state, which makes it trivially testable with synthetic traces and impossible to make non-deterministic by accident. The same purity holds for the hint rules and the report. Side-effecting wrappers, the file writes and the dialogs, are thin shells over these pure cores. This is why the system can be tested

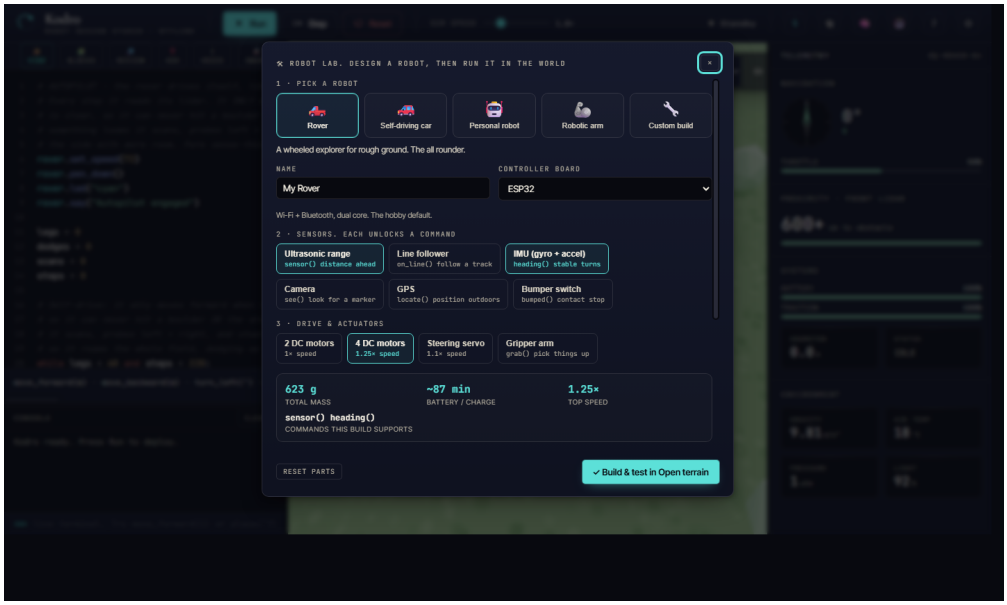

Figure 4.1. The Robot Lab. The user assembles a robot from a board, sensors and actuators, and the build's mass, battery life, top speed and supported commands update live. That specification then drives the simulation.

thoroughly without a display, which the evaluation chapter relies on.

4.6 Safety: the sandbox, the executor, and the gate

The user's program is untrusted, and the design treats it that way at three points. Before execution, the sandbox walks the program's syntax tree and rejects unsafe imports and builtins, the file opens, the process spawns, the attribute walks that reach for forbidden capabilities. During execution, the executor runs the program in a daemon thread under a hard wall-clock timeout, folding every failure mode, a syntax error, a runtime error, a sandbox violation, a timeout, into one structured result rather than letting an exception escape. After execution, the simulation gate is the run itself: the program drives the robot in a world and is scored, so a wrong program fails visibly on screen against the scenario rather than silently on real hardware.

These three points are the concrete form of the central analysis from Chapter 3. The model never touches any of them. It cannot relax the sandbox, it cannot extend the timeout, and it cannot decide that a program passed. The deterministic layer owns safety end to end, which is what lets the model be helpful without being dangerous.

4.7 Worlds, motion, and moving agents

The validation has to happen somewhere believable, so the design recommends a world to fit the robot. A self-driving car is dropped into the City, with one-way lane traffic that loops and brakes for the robot and pedestrians that keep to the pavements and use the crossing. A home or arm robot is dropped into the Room, with furniture and people. A rover is dropped onto a planetary terrain. Figure 4.2 shows the City world in three dimensions, which the user can orbit a full turn to read the scene from any angle.

The motion is designed to read as natural rather than to be physically exact. The robot does not slide like a prop; it transfers weight under acceleration, banks into turns, settles on its suspension, and steers from the front wheels where the chassis has them. The honest description of this, returned to in Chapter 7, is that it is a per-type kinematic model that fakes the feel

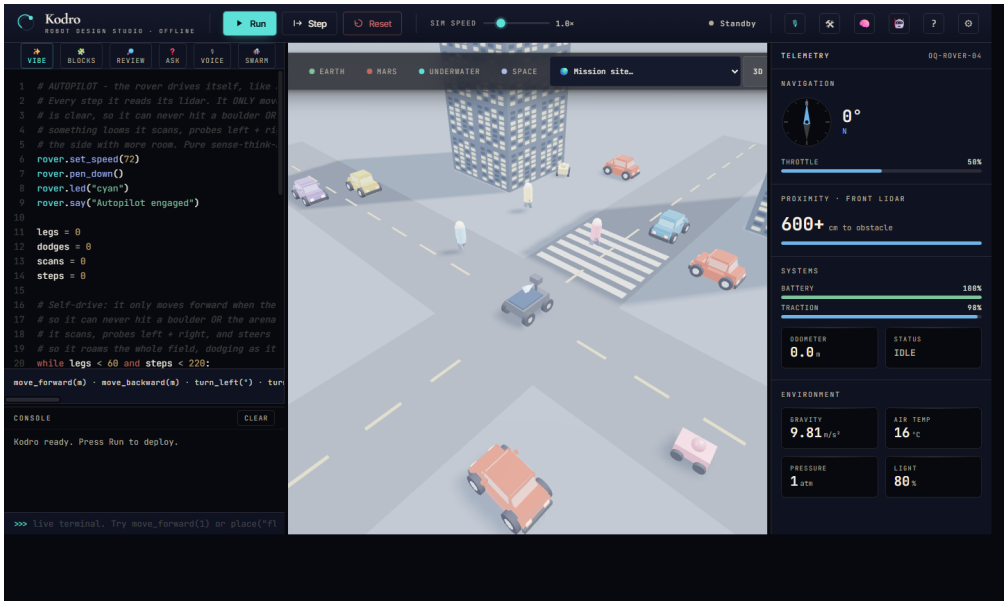

Figure 4.2. Validation in a realistic world. The designed robot runs in a city street rendered in three dimensions, among traffic and pedestrians that move around it, so its behaviour can be watched before any hardware is built.

convincingly rather than a rigid-body solver that resolves forces. This is a deliberate trade. The believable feel is what a non-expert needs to judge a behaviour, and it runs smoothly on a laptop, where a full solver would not.

4.8 The grounded assistant and its deterministic fallback

The assistant is designed around the principle that the model may generate but the validators decide. It is given the robot specification and the command registry as its grounding, and it is asked to suggest code only from that registry. When the user asks for a behaviour that needs a part the build lacks, the assistant is designed to refuse and to point back to the specification, explaining which part is missing, rather than to invent a call. An automatic code reviewer sits alongside it, proposing and then critiquing the model's output and accepting a rewrite only after it passes the same sandbox a manual run would.

Behind all of this is the deterministic fallback, which is what makes the offline guarantee real. If no model is installed, or if the model does not answer within a fixed time budget, a rule engine returns rule-based code and checks, and the platform stays fully usable. The fallback is not a degraded mode bolted on; it is the floor the whole assistant stands on, and the design treats the model as an optional accelerator above a guaranteed deterministic base.

4.9 Self-refinement without retraining

The final design element is the refinement loop, and the design is careful to call it refinement and not improvement, because nothing here changes the model's weights. Three local, countable mechanisms carry it, all in the local store. After each run the system writes a short reflection on what failed and what worked, and retrieves the most similar past reflections into the next suggestion. Routines that passed validation are kept as named, parameterised skills and composed into later designs. A record of past designs and their outcomes is sampled so that advice on a new build is anchored in what happened on similar ones. None of this edits the model; all of it is system-level memory, honestly framed, and the evaluation asks directly whether it produces the fall in iterations the objective requires.

Chapter 5

Implementation

This chapter describes how the design was realised. It is grounded in the codebase and in the running test-evidence log, and the engineering challenges are drawn from the real commit history, which makes them honest material for critical reflection rather than a tidy story told after the fact.

5.1 The program surface and the interpreter

The user's program runs through a Python subset interpreter, written in JavaScript and vendored into the bundle, which compiles a source string into a generator that yields motion and sensor events one at a time. The generator design is what lets the studio animate a run smoothly: the application pumps the generator, advancing the robot one event per tick, so the user sees the robot move rather than jumping to a final state. The interpreter supports the constructs a non-expert needs, sequence, selection, iteration with both `for` and `while`, functions, and the sensor reads, and it guards the pathological cases a beginner can stumble into, such as an enormous exponent, so that a slip produces a contained result rather than a hang.

A subtlety worth recording is that two call styles are both valid and both correct. A bare call such as `move_forward(2)` moves two metres, scaled internally to engine units, while a method call such as `rover.forward(200)` moves two hundred centimetres. This dual surface is convenient but it is also a known source of the arena-scale mismatch discussed in Chapter 7, and it is recorded honestly rather than hidden.

5.2 The robot specification and the derive step

The Robot Lab implements the single source of truth from the design. A catalogue of boards, sensors, actuators and chassis types backs a derive step that computes the structured specification from the chosen parts. The derive step is where the physical character of the robot is set: the mass is summed from the parts, the top speed and acceleration follow from the motor and the mass, the battery capacity and drain follow from the board and the load, and the supported-commands list is assembled from exactly the parts that provide each command. Saving a build dispatches an event carrying the recommended world, which the studio listens for, so designing a robot and being dropped into the right world to test it are one action.

5.3 The worlds and the natural-motion tick

The three-dimensional viewport is built on core Three.js with no asset loader, so every robot, obstacle, terrain feature and agent is procedural geometry generated in code, boxes, cylinders, spheres, extruded shapes and canvas textures, lit by an environment map with shadow mapping and tone mapping. The open-terrain ground is shaded rather than flat: a tileable normal map, derived by a Sobel pass over a periodic height field, and a roughness map are both generated in canvas at scene-build time, so the sun and fill light graze real micro-relief instead of a coloured plane. The City and the Room are built in code, and the moving agents, the traffic and the pedestrians, run in a shared coordinate space with one-way lanes, looping paths, and a rule that brakes them for the robot. The natural-motion tick is the code that gives each robot type its

feel: the per-type pitch under acceleration, the roll into a turn, the suspension settle, and the front-wheel steer, applied as a function of the robot's velocity and heading change each frame. The world is chosen from a catalogue of presets, the City and the Room alongside a Robotics Lab, a Warehouse, a Minimal Debug Grid and a set of real outdoor terrains, each carrying its own friction and lighting so the same program behaves differently across them. A render-quality control spanning Low to Cinematic bounds the two heaviest costs, the shadow resolution and the pixel ratio, so the scene stays smooth on a laptop with no discrete graphics card on the lower tiers and maxes its fidelity for a screenshot on the highest. The Cinematic tier additionally runs a small, hand-written, fully offline post chain, a bloom built from a bright-pass and a separable Gaussian blur composited additively over the base render together with a vignette, written directly against the core renderer because the library ships its post-processing only in an examples tree that is not vendored and cannot be fetched under the offline rule. The pass is gated so that it is disabled under a reduced-motion preference, turned off if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its targets, so the offline and low-hardware guarantees hold on every tier. A first-person toggle drops the camera onto the robot for a driver view.

5.4 Offline shading without an asset pipeline

The offline rule shapes the renderer as sharply as it shapes the model. The single largest lever for visual quality in a real-time scene is usually an asset pipeline: a glTF or GLB loader and a set of authored, well shaded models. That lever is closed here. Three.js ships its loaders and its post-processing only in an examples tree that is not part of the vendored core, and the zero-network constraint means neither the loader nor any quality interchange model can be fetched and committed; a hand-authored model would in any case be no better than the procedural geometry it would replace. The honest design response is to push the procedural surfaces as far as they will go and to be candid that an importer is future work rather than a missing feature that was skipped.

Two pieces of that work are worth recording because they were done under the constraint rather than around it. The first is surface relief. The open-terrain ground already carried a baked colour grain, but light still struck it as if it were glass, which was the strongest tell that the world was a tech demo. Each ground now generates, in canvas at scene-build time, a tileable normal map together with a roughness map. The normal map is a Sobel pass over a periodic height field whose waves are integer multiples of the tile size, so the derived normals are seamless under repeat wrapping; the result is that the existing physically based sun and fill light graze real micro-relief, and sand reads as sand, regolith as pits, the seabed as ripples. No texture file is loaded and no new binary is vendored.

The second is a post-processing pass for the highest quality tier. Because the library's bloom pass is not available offline, the pass is written directly against the core renderer: the proven base image is drawn to the canvas exactly as the lower tiers draw it, the scene is then rendered once more into an offscreen target used only as a bloom source, and a bright-pass, a separable Gaussian blur, an additive bloom composite and a vignette are layered on top. Drawing the bloom additively over the unchanged base, rather than replacing the image, means the worst case is that the scene simply looks like the normal render, which makes the feature safe to ship. It is gated to the Cinematic tier, disabled under a reduced-motion preference, turned off automatically if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its render targets or if the pass throws at frame time. The shading and the post pass are headless safe: with no document or WebGL context the generators return nothing and the scene renders exactly as before, so the offline bundle-render test never touches a canvas. This

is the offline constraint treated as a design input, not an apology.

5.5 Memory, the assistant, and the fallback

The self-refinement memory is implemented over the local store as reflections and saved skills, written after a run and retrieved before a suggestion. The assistant is implemented as a client to a local Ollama server, preferring a small coder model and grounding each request in the specification and the command registry, with an automatic reviewer that proposes and then critiques. The most important implementation detail is the guard: every networked or optional capability, the model, the audio, the achievement toasts, is best-effort and hard-guarded, so an absent server, a machine with no audio device, or a headless environment produces a silent no-op rather than an error. The deterministic rule engine sits underneath as the guaranteed path, and the switch to it is automatic on a missing model or a timeout.

5.6 Engineering discipline

Each unit of work closed only when a full gate passed: the test suite with branch coverage, the linter, the format check and a strict type check, after which the change was committed, pushed, and verified green on continuous integration. The coverage gate ratcheted upward as the codebase matured and settled at a minimum of eighty five percent, with the actual figure around eighty six. This discipline is itself a contribution, because the decision log and the test-evidence log it produced give the claims in this dissertation an audit trail that can be checked against the repository.

5.7 Representative listing

The scorer illustrates the trace-driven, pure-core principle from the design. It takes a scenario, a trace and the source, and returns a structured result with no input, no output and no hidden state, which is exactly what makes it testable with synthetic traces.

```
@dataclass(frozen=True, slots=True)
class GradeResult:
    passed: bool
    reasons: list[str]    # one user-facing line per failed criterion
    score: int            # 0 to 100

def grade(lesson: Lesson, tracer: Tracer, source: str = "") -> GradeResult:
    aggregates = _compute_aggregates(tracer.events())
    reasons: list[str] = []
    for criterion in lesson.success_criteria:
        reason = _check_criterion(criterion, aggregates, lesson, source)
        if reason is not None:
            reasons.append(reason)
    score = max(0, 100 - SCORE_PENALTY_PER_FAILURE * len(reasons))
    return GradeResult(passed=not reasons, reasons=reasons, score=score)
```

5.8 Two front-ends over one engine

The interface exists in two forms over the same engine. The original is a Tkinter interface that ships with Python and renders on any machine, retained as the guaranteed-offline fallback. After formative review found the Tkinter look dated for an audience that benchmarks software against web and mobile apps, a second front-end was added in the medium the reference design was authored in, a vendored offline React interface rendered in a pywebview window. Both front-

ends drive the same engine, scorer, library and store. The honest risk here, recorded in the next chapter, is the dual interpreter: the web interface ships its own JavaScript interpreter for instant animation, whose surface differs from the Python API, and the planned convergence is to stream real engine events to the view and retire the second interpreter.

5.9 Challenges met and fixed

These are real defects, found and fixed, and they make the strongest reflection points.

- **A procedural surface over a stateful engine.** A flat set of free functions has to reach a single live engine. This was solved with a module-level binding that the functions consult at call time, which also let the headless tests and the scorer drive the engine with no interface present.
- **An open loop.** An early build ran the user's program but never responded to it, with no pass or fail and no feedback. The fix wired scoring, hinting, persistence and the verdict into a single finish step at the end of every run, which a structured review had identified as the single biggest weakness of that build.
- **A silently detached trace.** While fixing the loop, the active trace could be detached between runs, so a run recorded zero events and some integration tests passed vacuously. The fix re-asserts the trace and engine bindings at the start of every world reset, and the tests were strengthened to prove real movement, since the battery only drains if the robot actually moved.
- **Headless interface fragility.** A modal error dialog blocked forever on the headless test runners, once burning a job to the six-hour ceiling. A per-test timeout was added so any hang fails fast with a traceback, which then pinpointed further headless issues and led to treating one platform's interface tests as informational while the other two remain hard gates.

5.10 Packaging

The desktop binary is built with PyInstaller, which bundles the interpreter, the runtime, the libraries and the asset bundle into a single windowed application, so deployment is one download with no separate runtime install. The build is reproducible from a specification file and was smoke-tested on a real desktop, where the packaged application starts cleanly and renders the full interface, which the headless suite otherwise exercises without a display.

Chapter 6

Evaluation

This chapter records how Kodro was evaluated. It separates what was actually measured, the automated verification and an analyst-run formative review, from what is planned but not yet done, a study with real users. Nothing here reports data from human subjects, because that study has deliberately been left for a point at which it can be run properly under consent.

6.1 Evaluation strategy

Three complementary methods were chosen, in increasing cost and decreasing frequency.

Method	What it checks	Status
Automated test suite	Correctness of every subsystem, regression safety	Done, continuous
Functional QA harness	Interpreter and kinematics behaviour end to end	Done, continuous
Persona review	Whole-product usability across user types	Done, formative
User and refinement study	Real efficacy and the refinement loop	Planned, future work

Table 6.1. The three-method evaluation strategy and the status of each.

6.2 Automated verification

The primary evidence of correctness is the automated suite, which grew with the codebase and gated every change. The Python engine carries more than eight hundred tests, unit, property-based and headless integration, at around eighty six percent line and branch coverage, gated at a minimum of eighty five percent, run on continuous integration on every push. These tests pin the behaviour that matters: the procedural API never raises and clamps bad input; the physics, the sensors and the scorer are deterministic; the sandbox rejects unsafe code and enforces a hard timeout; and the end-to-end run, score, hint, persist and reward loop is exercised on the fully wired application, so the loop is proven to close rather than asserted to.

Alongside the Python suite, a separate functional quality harness drives the shipped interpreter with the real kinematics and the wall ray, and asserts the command semantics and every bundled example program. It passes at twenty one of twenty one. Every example terminates, moves, stays inside the arena, never hits a wall and never throws, and the command semantics, the two motion call styles, the turn, the speed clamp, the guarded huge exponent, the loops and the sensor reads, are each asserted. This harness is the evidence that the user-facing program surface behaves, independently of the Python engine.

A third method complements these: a structured adversarial review of the interactive surface, which the automated suites by design do not reach. The review fans out one reviewer per dimension, first run, command gating, movement, accessibility, performance and small-screen layout, and then has each candidate defect independently re-checked against the code before

it is accepted, so that plausible but wrong findings are filtered out rather than acted on. It surfaced defects the unit tests could not, because they live in the wiring rather than in a pure function: selecting a real-world site resolved through the wrong table and crashed the view; the three-dimensional robot mesh faced ninety degrees off its travel direction while the physics stayed correct; the parts catalogue advertised commands the interpreter does not implement; the reduced-motion path sampled a long move too coarsely to catch a small obstacle; and several controls fell below the accessible-contrast and touch-target thresholds. Each confirmed critical and high-severity finding was fixed and then re-verified, the visual ones in a headless browser capturing the real WebGL render through a software rasteriser so the proof needs no graphics card and no network. The value of the method is precisely that it targets the integration seams the deterministic suites leave uncovered.

6.3 Persona review (formative)

To evaluate the whole product rather than its units, a structured review was conducted across simulated personas spanning the range of makers who might use the tool. Each persona drove a session and returned a mark out of ten with the concrete defects it found; after each round the named defects were fixed and the build was re-rated. Table 6.2 reports the measured trajectory.

Round	Personas	Focus	Mean (/10)
1	50	First contact with the build	5.86
2	12	After the first round of fixes	6.50
3	50	After wiring the model and tools into the app	7.10
4	50	After grounded answers and the voice route were added	7.36
5	8	Focused review of the new three-dimensional world	6.40

Table 6.2. Persona review trajectory across rounds. The figures are reported as measured by the analyst-run method, with the threats to validity stated below.

6.4 Threats to validity

The persona review is a low-cost inspection method in the tradition of heuristic evaluation, and it is reported with its limits stated plainly rather than dressed up as user evidence. Three threats matter. First, a single analyst simulated every persona, so the scores carry that analyst's bias and the inter-rater reliability is unknown. Second, the personas are informed guesses about real makers and cannot capture genuine confusion, motivation or the messiness of real use. Third, re-scoring a build the analyst made risks optimism, so a rising mean should be read as a direction of travel that the tool became more learnable to drive, which is a usability signal and nothing more. The fifth round, scoring the new three-dimensional world lower than the fourth, is left in deliberately, because the honest record includes the rounds that went down as well as the rounds that went up. These threats are exactly what the planned user study is designed to address.

6.5 The planned studies

Three studies are specified and left for a point at which they can be run under consent. The first asks whether the tool helps: a within-subject design with about sixteen non-experts, each attempting two comparable held-out scenarios, one with Kodro's grounded help and reviewer and one without, with order counterbalanced, where success is a design that completes the scenario in

at least sixteen of twenty randomised runs, and iterations and collisions are recorded alongside. The second asks whether the tool refines: the same participant tackles a sequence of related scenarios in two arms whose only difference is the memory, the full arm against an ablated arm with reflection retrieval and the skill library turned off, with the full arm expected to reach a working design in at least a quarter fewer iterations. The third measures the assistant directly on a fixed set of design questions including adversarial and off-specification ones, where grounded answers must be correct at least eighty five percent of the time and invent a missing part less than five percent of the time, and where grounding must beat the same model asked without the specification, with the five percent invention rate set as a release gate above which only the deterministic path ships. The analysis for each is fixed before any data is collected, and a positive result is to be read as a signal to confirm at scale rather than as proof, because the samples are small and convenient.

6.6 Instrumentation

The application is already instrumented for these studies without any cloud service. Every run is recorded with its program, trace, score, battery and collisions, the reflections and skills accumulate in the local store, and the results can be exported on demand, all on device. This is consistent with the offline constraint and it simplifies the ethics and data-protection position for any future study, because no personal data ever leaves the machine.

6.7 Summary of the evaluation

The honest position is that the product has been verified, it does what it claims and the tests prove it, and it has been heuristically reviewed, it should work for a range of makers, but it has not yet been validated with real users. The interpreter passes at twenty one of twenty one, the engine suite passes at scale with high coverage, and the persona review traces a real improvement as defects were fixed. The remaining step to a confident claim of usefulness is the human study, for which the instrumentation is in place, and the next chapter is candid about the limitations that study and the roadmap will have to address.

Chapter 7

Discussion and Limitations

A dissertation that only reports what works is not trustworthy. This chapter sets out, plainly, what Kodro does not do yet, so that a reader and a future contributor can plan around the gaps. Each limitation is framed as something the design is working toward rather than something broken with no path forward, and each is labelled by honest status: working with a known gap, experimental, or roadmap.

7.1 Motion is kinematic, not rigid body

The most important limitation to be clear about is the one most easily oversold. The robot and every moving agent advance through a hand-written kinematic tick. There is no rigid-body solver, no contact forces and no friction model resolving the motion in the runtime. A car banks and a rover throws its weight around because the per-type motion code fakes the feel convincingly, not because a physics engine resolves forces, and collisions are detected as overlap events and recorded rather than resolved with impulses. This is a deliberate trade that buys a believable feel at a laptop-friendly cost, and it is the right trade for a non-expert judging a behaviour. It is not the right trade for deployment-grade validation, and the dissertation does not claim it is. The roadmap item that addresses this is the wiring of a real rigid-body engine into the production runtime.

7.2 The physics engine exists but is not on the hot path

Related to the above, the Python engine ships a tested wrapper around a real two-dimensional physics library that builds boundary walls, places the robot and obstacles, and records collisions. It is exercised by the Python suite and used by the Python scorer. It is not, however, what the visible studio runs, because the shipped desktop interface steps the world through the vendored kinematic interpreter. So the rigid-body path is real code, but it is not on the hot path the user sees when they press run. Making that wrapper the source of truth for the visible motion is the single largest change on the roadmap, because it touches the interpreter, the viewport and the scorer at once.

7.3 Sensor gating is partial

The grounding guarantee that the design rests on is fully enforced today only for the ultrasonic and distance sensors. The other sensors, the inertial unit, the positioning sensor, the camera and the bumper, exist in the interpreter and the API but are not yet gated against the specification end to end, so in some cases the assistant could suggest a call to a sensor the chosen build does not carry. This is the gap between the design's promise and the current code, and it is named here rather than glossed, because the whole safety argument depends on closing it. Full gating across every surface is a near-term roadmap item, and it is the first thing the realism work should finish.

7.4 Procedural geometry, with surface shading and a gated post pass

The viewport has no asset loader for any interchange format, so it cannot import a real robot description or a modelled part; everything is procedural geometry. This part of the limitation is genuine and stands. The shading side, however, is now mitigated rather than absolute. Each open-terrain ground builds a tileable normal map, derived by a Sobel pass over a periodic height field, together with a roughness map, both generated in canvas at scene-build time, so the light grazes real micro-relief instead of a flat coloured plane. The Cinematic quality tier adds a small, hand-written, fully offline post chain, a bloom built from a bright-pass and a separable Gaussian blur composited additively over the base render, with a vignette. It is written directly against the core renderer because the library ships its post-processing only in an examples tree that is not vendored and cannot be fetched under the offline rule, and it is gated to the Cinematic tier, disabled under a reduced-motion preference, turned off automatically if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its targets, so the offline and low-hardware guarantees are not weakened. What remains genuinely absent is a glTF or similar asset loader and the heavier effects, ambient occlusion, depth of field and motion blur. The loader stays on the roadmap rather than shipped because neither it nor good interchange models can be obtained under the zero-network constraint in this environment, and that is recorded as future work rather than presented as a gap that was quietly skipped.

7.5 An arena-scale mismatch between the two engines

The in-browser interpreter runs the robot in a thirty-metre arena while the Python engine and scorer work in a ten-metre world. Every bundled example runs and the conformance check passes, because the criteria are scale tolerant, but a program tuned visually in the web interface can score differently under the Python scorer. The metres-to-centimetres scaling in the interpreter mitigates this for motion but not for boundary behaviour. A single shared arena definition that both engines read is the fix, and it is recorded as a known issue rather than left for a reader to discover.

7.6 Voice depends on the platform without an optional component

The voice route has two paths. With an optional offline speech component installed it transcribes on any operating system; without it the route falls back to the platform speech recogniser, which is available only on one operating system and depends on a language pack. The deterministic phrase parser that turns a transcript into a robot line works everywhere once it has text, so the limitation is in the speech-to-text step and not in the command mapping. Making the cross-platform component the default path is a small, well-understood roadmap item.

7.7 The small-model ceiling

Finally, the honest limit named throughout returns here. The local model is small by necessity, and a small model cannot match a frontier cloud model on raw capability. The design answers this not by pretending otherwise but by leaning on grounding and the simulation gate, so that the system is safe even when the model is weak, and useful because the help is anchored in the build rather than in the model's cleverness. The cost is that the assistant is a competent drafter and not an expert, and the deterministic path, not the model, is what the offline guarantee and the safety argument actually rest on.

Chapter 8

Conclusion and Future Work

8.1 Summary

This project set out to build a single, offline, free desktop platform on which a capable non-expert designs a custom robot, programs it three ways against one meaning, and validates its behaviour in a realistic, agent-populated simulation before building any hardware, with a local model that helps without ever having the last word on safety. Against the core objectives, the artefact delivers a specification that drives the simulation so that changing the build changes the behaviour, a sandboxed execution and scoring path, a validation layer with moving agents and randomised runs, a grounded assistant with a guaranteed deterministic fallback, and a self-refinement memory of reflections and skills, all on one machine with the network off. The interpreter passes its functional harness at twenty one of twenty one, the engine suite passes at scale with around eighty six percent coverage, and the persona review traces a real, measured improvement as defects were fixed.

8.2 Reflection

Three honest reflections stand out. What worked was building scoring, hinting and replay on one trace abstraction and keeping the decision cores pure, which paid off repeatedly in testability and in the ease of adding features late. What was harder than expected was headless interface testing across operating systems, where the honest outcome was to gate hard on the stable platforms and treat the fragile one as informational rather than to weaken the tests. What I would do differently is to wire the feedback loop first rather than last, because the most important behaviour, the system responding to a run, was implemented after much of the surrounding interface, and discovering its absence through a structured review rather than by design cost rework.

8.3 Future work

The roadmap follows directly from the limitations, ordered roughly by the order the work should be taken on.

1. **Full sensor gating** across every surface, so the inertial unit, the positioning sensor, the camera and the bumper are gated exactly as the ultrasonic sensor is. This closes the gap in the safety argument and is the first priority.
2. **Rigid-body physics on the hot path**, making the tested physics wrapper the source of truth for the visible motion, so the viewport reflects real contact forces, friction and collision response.
3. **A single shared arena definition** so the two engines agree on the world size and a program scores the same way it looked.
4. **Richer domain randomisation**, varying obstacle placement, lighting, friction and sensor noise across runs so that a passing program is one that generalises rather than one that memorised a layout.
5. **A model-selection interface** that lists the models the local runtime reports as installed and lets the user choose the drafter and the reviewer independently, making the offline assistant

legible rather than implicit.

6. **An asset and import workflow**, a documented path for authoring parts and terrain and loading them, and an importer for standard robot descriptions, so published robot models can be used directly.
7. **A research bridge** to the established heavyweight simulators, exporting a Kodro build and program for high-fidelity validation and pulling the results back, which keeps Kodro light while opening a path to rigorous physics when a user needs it.
8. **A bridge to a robotics middleware** as an optional, separate component that talks to a running daemon over a local socket, so a validated Kodro program can drive real topics, recorded honestly as a long way out because it fights the offline guarantee.
9. **The human studies** specified in the evaluation, the usability study, the help study and the refinement-ablation study, run under consent with the instrumentation that is already in place.

8.4 Closing statement

The originality of this work is not a new algorithm. Retrieval grounding, a sandboxed gate, an experience memory, a skill library and domain randomisation are each known, and the offline constraint actively limits how clever any one of them can be. The contribution is the demonstration that these parts combine into an honest design and validation platform that runs entirely on one machine, and a measured, candid account of how far offline self-refinement can go when changing the model weights is off the table. The constraint is the point of the project, not a limitation to apologise for, and the remaining step to a confident claim of usefulness is the human study for which the tool is already built and instrumented.

Bibliography

Ahn, M., Brohan, A., Brown, N. et al. (2022) Do as I can, not as I say: grounding language in robotic affordances. *arXiv:2204.01691*.

Cemri, M., Pan, M.Z., Yang, S. et al. (2025) Why do multi-agent LLM systems fail? *arXiv:2503.13657*.

dos Santos, V.G., Khadraoui, I., Farhat, I. et al. (2026) ALRM: agentic LLM for robotic manipulation. *arXiv:2601.19510*.

Huang, L., Yu, W., Ma, W. et al. (2023) A survey on hallucination in large language models. *ACM Transactions on Information Systems* (arXiv:2311.05232).

Khati, D., Rodriguez-Cardenas, D., Pantzer, P. and Poshyvanyk, D. (2026) Detecting and correcting hallucinations in LLM-generated code via deterministic AST analysis. *arXiv:2601.19106*.

Lee, Y., Song, J.Y., Kim, D. et al. (2025) Hallucination by code generation LLMs: taxonomy, benchmarks, mitigation, and challenges. *arXiv:2504.20799*.

Lewis, P., Perez, E., Piktus, A. et al. (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, pp. 9459 to 9474.

Liang, J., Huang, W., Xia, F. et al. (2023) Code as policies: language model programs for embodied control. *IEEE International Conference on Robotics and Automation*.

Papert, S. (1980) *Mindstorms: children, computers, and powerful ideas*. New York: Basic Books.

Reza, Z., Mazur, A., Dugdale, M.T. and Ray-Chaudhuri, R. (2025) Small models, big support: a local LLM framework for educator-centric content creation and assessment with RAG and CAG. *arXiv:2506.05925*.

Shinn, N., Cassano, F., Berman, E. et al. (2023) Reflexion: language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36 (arXiv:2303.11366).

Sun, T., Xu, J., Li, Y. et al. (2025) BitsAI-CR: automated code review via LLM in practice. *arXiv:2501.15134*.

Tao, Z., Lin, T.E., Chen, X. et al. (2024) A survey on self-evolution of large language models. *arXiv:2404.14387*.

Tobin, J., Fong, R., Ray, A. et al. (2017) Domain randomization for transferring deep neural networks from simulation to the real world. *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 23 to 30.

Wang, G., Xie, Y., Jiang, Y. et al. (2023) Voyager: an open-ended embodied agent with large language models. *arXiv:2305.16291*.

Wang, W., Li, Y., Jiao, L. and Yuan, J. (2026) Ro-SLM: onboard small language models for robot task planning and operation code generation. *arXiv:2604.10929*.

Wu, R., Wang, X., Mei, J. et al. (2025) EvolveR: self-evolving LLM agents through an experience-driven lifecycle. *arXiv:2510.16079*.

Chapter A

Glossary and Abbreviations

Definitions are scoped to how each term is used in this dissertation.

Term	Meaning
API	Application programming interface; here the small procedural command surface the user's program calls.
AST	Abstract syntax tree; used by the sandbox to reject unsafe code and by the scorer to check required constructs.
Domain randomisation	Varying friction, mass, sensor noise and the scene across runs so behaviour that survives the spread is likely to survive reality.
Grounding	Constraining the model's suggestions to the build the user assembled, so it cannot call a part that is not fitted.
Kinematic motion	Motion advanced by a hand-written tick that fakes the feel of weight and traction, as distinct from a rigid-body solver that resolves forces.
Ollama	A local model runtime; the optional, loopback-only backend for the assistant.
RobotSpec	The structured robot specification derived from the chosen parts; the single source of truth for behaviour and the command registry.
Sandbox	The pre-execution check that rejects unsafe imports, builtins and operations before a program runs.
SQLite	The embedded, file-based database used for the single local store.
Trace	The recorded sequence of command events from one run; the single representation that scoring, hinting and replay all read.

Table A.1. Glossary of terms as used in this dissertation.

Chapter B

The Command Surface

The procedural command surface the user programs against is a small set of plainly named functions, gated by the build. The motion and query commands below are representative; a command is present only when the part it depends on is fitted.

```
move_forward(distance)       # drive forward; metres in the bare style
move_backward(distance)      # drive backward
turn_left(degrees)           # turn in place to the left
turn_right(degrees)          # turn in place to the right
set_speed(percent)           # cap speed; clamped by motor and mass
stop()                       # halt
read_distance()              # only if an ultrasonic or ranging sensor is fitted
read_heading()              # only if an inertial unit is fitted
collect_sample()             # only if a collector actuator is fitted
log(message)                 # write a line to the console
```

Chapter C

A First Program

A short program that drives a square and leaves a trail, used as a first example. It calls only commands the default rover carries.

```
# A rover that drives a square and leaves a trail.
set_speed(60)
for side in range(4):
    move_forward(2)
    turn_right(90)
```

Pressing run animates the rover along the path with weight transfer and a draining battery; pressing step advances one event at a time. Changing the build, a heavier chassis or a weaker motor, changes how the same program behaves, which is the point the whole platform is built to make.